

Deployment runbook

Ordered build sequence synthesizing every doc/config in this repo into one checklist. Each step links to the doc with the actual detail — this is a map, not a duplicate.

Phase 0 — Before touching anything

- ☐ **First:** on one real ATEM at the event frame rate and firmware, measure a camera ISO file, a laptop/slides ISO file and the program file (`ffprobe` , or `size ÷ duration`) against the planning figures (camera 35 / 45 / 70 Mbps, laptop 15 / 25 / 40 *estimated*, program ~10) — they set the live ISO scope (`ATEM_ISO_INPUTS`) and the 8 TB pool sizing — `docs/open-questions.md` #0
- ☐ Confirm the consolidated server's full spec (validated ECC, PCIe lanes for the NVMe pools, RAM/cores — no GPU/IOMMU checks needed any more) — `docs/open-questions.md` #1
- ☐ Confirm the Cloud Gateway model — LAG support (assumed, not confirmed) and its real inter-VLAN routing throughput — `docs/open-questions.md` #17
- ☐ Benchmark Tailscale throughput on one Slate AX (GL-AXT1800) with `iperf3 --bidir` , on a direct path (not DERP), **before buying all 14** — target above ~160 Mbps combined; the ~150–250 Mbps in the bandwidth model is an estimate — `docs/open-questions.md` #19
- ☐ Decide a real hostname/DDNS for the DERP server and confirm you can port-forward on the Cloud Gateway — `docs/tailscale.md`
- ☐ Confirm VMix's actual recording mode/bitrate and get SMB sharing set up on all 4 VMix PCs — `docs/vmix-record-ingest.md`
- ☐ Confirm ATEM FTP credentials and recording path on a real unit — `docs/atem-iso-ingest.md`
- ☐ Decide a real hostname/DDNS for GLKVM-Cloud, and note the venue's public IPv4 — `GLKVM_ACCESS_IP` takes the IP, not the hostname — `docs/glkvms-cloud.md`
- ☐ Confirm ATEM Overseer's, ATEM Fleet Admin's, and Flock's real container images, and whether the ATEM and BirdDog Play can actually originate their monitoring/preview streams alongside their primary feeds — `docs/open-questions.md` #11
- ☐ Settle the live-editing open questions — footage-volume sizing, the Mac mini's combined Project Server + Remote Render role, standalone cameras in scope or not — `docs/open-questions.md` #12-14

Phase 1 — Mothership networking (Cloud Gateway)

1. Apply `config/unifi/network-config.yaml` via the UniFi Network UI/API — 4 uplink VLANs, firewall baseline, LAG port profile, DERP + GLKVM-Cloud port forwards. Steps in `config/unifi/README.md` .
2. Wire the 4 VLAN uplink cable runs (3 theatres per group) — `docs/topology.md` .
3. Bond the consolidated server's 2 NICs (802.3ad/LACP) into the Cloud Gateway (or an intermediate switch if LAG isn't supported) — `docs/topology.md` , watch the LACP rate/hash-policy gotcha with Ubiquiti gear.
4. Configure and verify the gateway's **static routes** — all 14 theatre/VMix /24 s (`192.168.2-13.0/24` , `192.168.20.0/24` , `192.168.21.0/24`) via `192.168.1.2` , per the `static_routes` block in

`config/unifi/network-config.yaml` — and the uplink-VLAN allow for `192.168.1.2:41641/udp`. Check the routing table in the UniFi UI lists all 14; once Phase 3 is done, `traceroute` to a theatre ATEM from a ready-room PC should show `192.168.1.1` then `192.168.1.2`. These routes are how the macvlan containers and the BirdDog Central VM reach the tailnet; that traffic crosses the server's bond twice (see `docs/server-specification.md` 's NIC section).

Phase 2 — Consolidated services server (Unraid)

1. Install Unraid (Unleashed or Lifetime licence — the pools exceed Starter's 6-device limit, see `docs/open-questions.md`). In Settings → Docker, turn on "Host access to custom networks" — the host has to hand tailnet replies back to the macvlan containers (`docs/open-questions.md` #18).
2. Create the BirdDog Central Windows VM — the design's only VM, no GPU passthrough needed — `docs/topology.md`. Recommended: give it its own persistent routes to the theatre /24 s via `192.168.1.2` (`route -p add`), so its NDI-fallback traffic doesn't hairpin through the gateway and cross the bond twice.
3. Bring up every Docker container in one shot with `config/docker-compose.yml` — Nextcloud, Restreamer, NDI Discovery Server, DERP, ATEM ISO Ingest, VMix Record Ingest, UniFi Controller, GLKVM-Cloud (`rttys + coturn`), ATEM Overseer, ATEM Fleet Admin, Flock, Tailscale subnet router. See `config/README.md` for the `.env` setup and prerequisites first — DERP and GLKVM-Cloud specifically need their hostnames/ port-forwards from Phase 0/1 in place before they're actually useful, and ATEM Overseer/Fleet Admin/Flock need their real container images confirmed first (see `docs/open-questions.md` #11) — DERP and GLKVM-Cloud will start but be non-functional without their hostnames/forwards, and the three placeholder-image services won't start at all until real images are filled in. Two services aren't plain pulls: `derp-server` is built from source on first `up`, and GLKVM-Cloud needs an upstream `gl-inet/glkvcloud` checkout for its endpoint and templates (`git clone https://github.com/gl-inet/glkvcloud.git` into `GLKVM_UPSTREAM`).
4. Optionally adopt the Cloud Gateway into the UniFi Controller for local management — `config/unifi/README.md`. Purely a local console; the network config in Phase 1 doesn't depend on it.
5. Router registration into GLKVM-Cloud happens later, in Phase 4, once the routers themselves exist.

Phase 3 — Tailscale tailnet

1. Load `config/tailscale-acl.json` into the tailnet admin console — cross-theatre isolation ACLs + the self-hosted DERP region.
2. Bring up the mothership's subnet router — the `tailscale-router` container applies the same flags as the mothership line in `config/tailscale-up-all-devices.sh` itself (routes, `tag:mothership`, `--accept-routes`, port 41641), so this is just a valid `TAILSCALE_AUTHKEY` pre-tagged `tag:mothership`. Confirm the ACL allows the subnet CIDRs, not just the tags (`docs/open-questions.md` #18).
3. Approve routes in the admin console (or configure `autoApprovers`) — nothing routes until approved, regardless of what's advertised.

Phase 4 — Theatre + VMix node routers (GL-iNet Slate AX / GL-AXT1800 ×14)

1. Flash/reset all 14 units, apply Wi-Fi lockdown per venue policy (not covered by the generated configs).
2. Apply each theatre's UCI config from `config/gl-inet/` — replace every `REPLACE_WITH_MAC_nn` with real device MACs first (config won't do anything useful otherwise). Steps in `config/gl-inet/README.md`.
3. Run each router's `tailscale up` line from `config/tailscale-up-all-devices.sh`.
4. Confirm the physical wiring matches the current plan: ATEM on the dedicated LAN 1 port, everything else (including BirdDog Play) via the Netgear switch on LAN 2 — `docs/topology.md`.
5. Register each router against GLKVM-Cloud (brought up in Phase 2) using the connection script from its web UI, run over SSH on each Slate AX — rides the tailnet already set up in step 3, no WAN exposure needed for this part — `docs/glkvm-cloud.md`.

Phase 5 — BirdDog Play + NDI discovery

1. Change the default `birddog` admin password on all 12 PLAY units.
2. Point BirdDog Central, all 12 PLAY units, *and both VMix node PCs' NDI config* at the NDI Discovery Server (`192.168.1.15:5959`) — BirdUI's Network panel on the PLAYS, NDI Access Manager on the Windows machines — `docs/streaming-flow.md`.
3. Set the SRT payload size to **1128 bytes** on every VMix node PC's SRT output and on Restreamer's outputs — the 1316-byte default doesn't fit Tailscale's 1280-byte MTU — `docs/open-questions.md` #20.

Phase 5b — ATEMs (all 12)

1. Patch every ATEM to the **common theatre input map** — identical on all 12: input 1 camera, 2 presenter laptop 1 (PowerPoint Main), 3 laptop 2 (VT Main / second presenter), 4–8 backups/spare — `docs/atem-iso-ingest.md`. The live ingest selects inputs by number fleet-wide, so a theatre patched differently pulls the wrong source.
2. Set each ATEM's **Streaming/record quality** so the program recording is **~8–10 Mbps** (ISO bitrates are fixed by frame rate and can't be set). Check one recorded program file's bitrate.
3. On one unit, record a short test and confirm the FTP folder layout and that ISO files carry `CAM <n>` in their names — `docs/open-questions.md` #21, #22.

Phase 6 — Nextcloud external storage + ingest pipelines

1. Run `config/atem-iso-ingest/setup-nextcloud-external-storage.sh` and `config/vmix-record-ingest/setup-nextcloud-external-storage.sh` — mounts the External Storage folders and prints the cron lines for targeted rescans. Add those cron entries.
2. Fill in real credentials, set `ATEM_ISO_INPUTS` in `.env` (default `1` = camera + program; `1,2` / `1,2,3` add the laptop inputs, for all 12 theatres at once, only if router and mothership-NIC headroom allow — see `docs/atem-iso-ingest.md` § Bandwidth), and start `pull-iso.py` (or the `rclone-mount-`

- `rsync/` alternative, which reads the same variable) — `config/atem-iso-ingest/README.md`. Inputs not pulled live stay on each ATEM's SSD — schedule the physical DIT offload of those drives (`docs/live-editing.md`).
3. Fill in real SMB credentials/share names and start `mount-and-sync.sh` — `config/vmix-record-ingest/README.md`.
 4. Configure Nextcloud version-retention for both ingest folders — both ingest docs flag this as worth tuning once running for real.
 5. **Dual-write caveat:** the second write destination (the edit-suite NAS, see `docs/live-editing.md`) is documented but **not yet implemented** in either ingest script — implement it or consciously defer it before the event (`docs/open-questions.md` #15). As shipped, both scripts write only the Nextcloud destination.

Phase 6b — Live editing subsystem

The edit suite (`docs/live-editing.md`) is its own 10GbE LAN, off the tailnet, so it can be built any time after Phase 2 — it only depends on the ingest pipelines (Phase 6) for its footage feed:

1. Stand up the `192.168.22.0/24` 10GbE LAN — switch, edit-suite NAS (`.22.2`), routed connection back to the mothership LAN for the dual-write (no Tailscale).
2. Set up the Mac mini (`.22.20`): install DaVinci Resolve Studio + the Project Server app, create the shared project library, enable it as a Remote Render node.
3. Connect both MacBook Pros (`.22.11` / `.22.12`) via Thunderbolt-to-10GbE, mount the NAS on all three machines with identical paths (Resolve's Remote Render requires the media volume mounted on every machine).
4. Run `resolve-configurator` against the event's session CSV to build the shell project; apply each smart-bin recipe once, by hand, in Resolve's UI.

Phase 7 — Verify before the event

- ☐ `tailscale status` / `tailscale ping` from a few nodes — confirm direct vs relay — `docs/tailscale.md`
- ☐ `tailscale netcheck` on a theatre router — confirm the self-hosted DERP region is reachable and not blocked by the uplink-VLAN firewall rule — `docs/open-questions.md` #3
- ☐ Copy an ATEM file mid-recording and confirm it opens/scrubs — empirically verify the "safe to read mid-write" assumption — `docs/atem-iso-ingest.md`
- ☐ From inside a macvlan container, reach a theatre device and get an answer back — e.g. `docker exec atem-iso-ingest python -c "import socket; socket.create_connection(('192.168.2.2', 21), 5)"` — proves the gateway static route, the host return leg and the subnet ACLs together (`docs/open-questions.md` #18)
- ☐ Confirm ingested files actually appear in Nextcloud's UI (not just on disk) for both pipelines
- ☐ Confirm cross-theatre isolation: from one theatre's subnet, verify you cannot reach another theatre's devices, only the mothership — this is the core ACL guarantee the whole design depends on (`config/tailscale-acl.json`)
- ☐ From off-venue (a network that isn't the mothership's own), confirm the GLKVM-Cloud web UI is actually reachable through `https://kvm.example.net:8443`, and that all 14 routers show up registered

and their SSH terminal/web-proxy access works — [docs/glkvm-cloud.md](#)

☐ Confirm all 12 theatres' monitoring/preview streams actually reach ATEM Overseer and Flock and show up live in each dashboard — this is the first real-hardware test of the dual-stream assumption flagged in [docs/open-questions.md](#) #11

☐ Live editing: confirm footage lands on the edit-suite NAS as it's ingested (once the dual-write is implemented — see Phase 6 step 5), both editors can open the shared show project simultaneously, the smart bins auto-fill as clips arrive, and a test export dispatched to the Mac mini via Remote Render completes — [docs/live-editing.md](#)

☐ Rehearse an NDI fallback end-to-end: pause the source node's VMix record ingest, confirm the node's VMix NDI output appears via the Discovery Server, confirm BirdDog Central picks it up and **re-originates** it (the PLAY must connect to Central, not directly to the node — direct would bypass the ACL model and the bandwidth math), switch one theatre's PLAY over via Flock, and measure the node uplink against the modelled ~130 Mbps — [docs/streaming-flow.md](#) , [docs/open-questions.md](#) #16